Probabilistic model checking

Tomaž Jonatan Leonardis, Janez Ignacij Jereb, Sonja Šuković, Vanja Stojanović, Martin Žust

Motivation

We want to model a probabilistic system and check its properties.

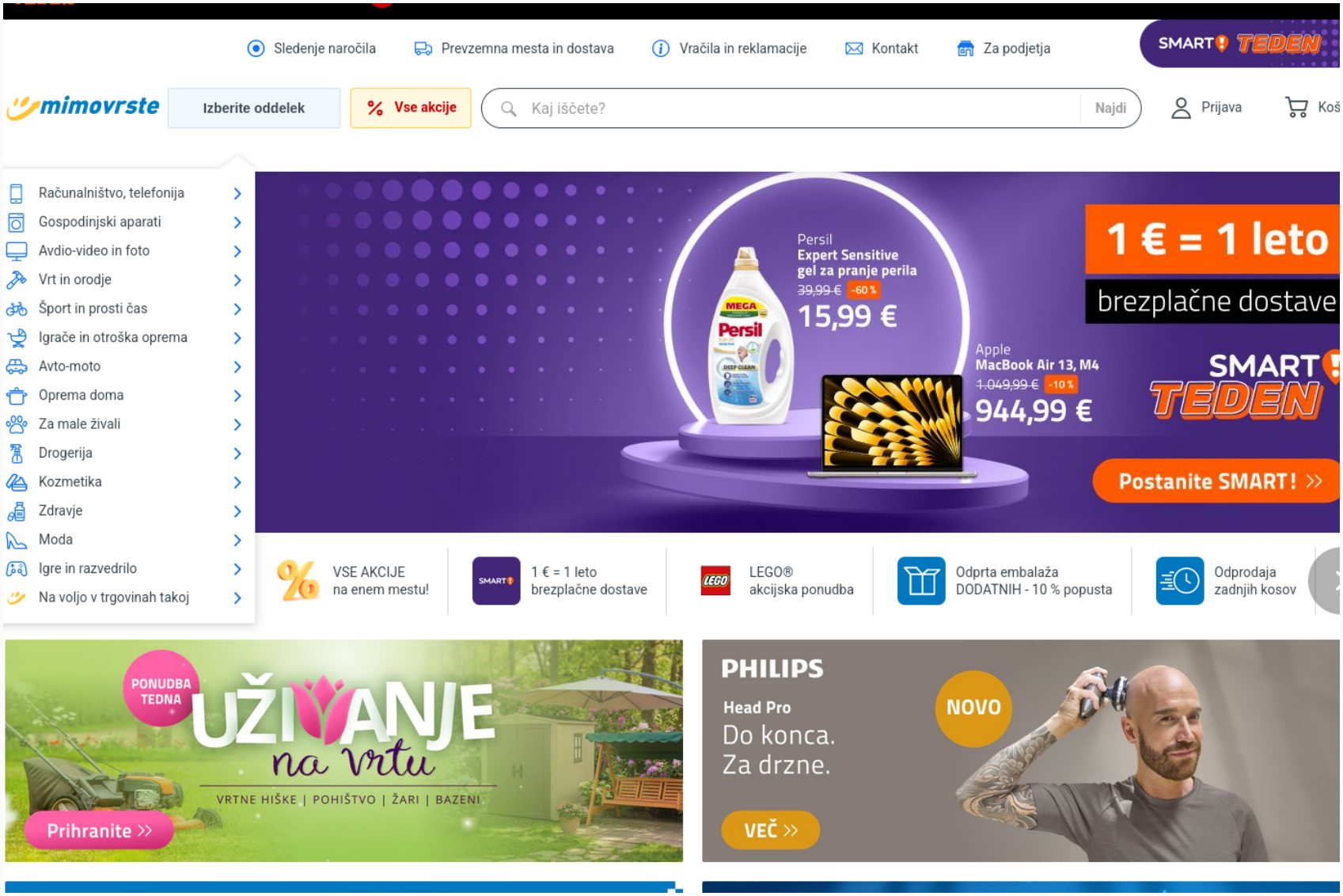
Examples include:

- Randomised distributed algorithms
- Communication, network and multimedia protocols
- Security protocols
- Biological systems
- Planning and synthesis
- Performance and reliability
- Game theory
- Power management
- ...

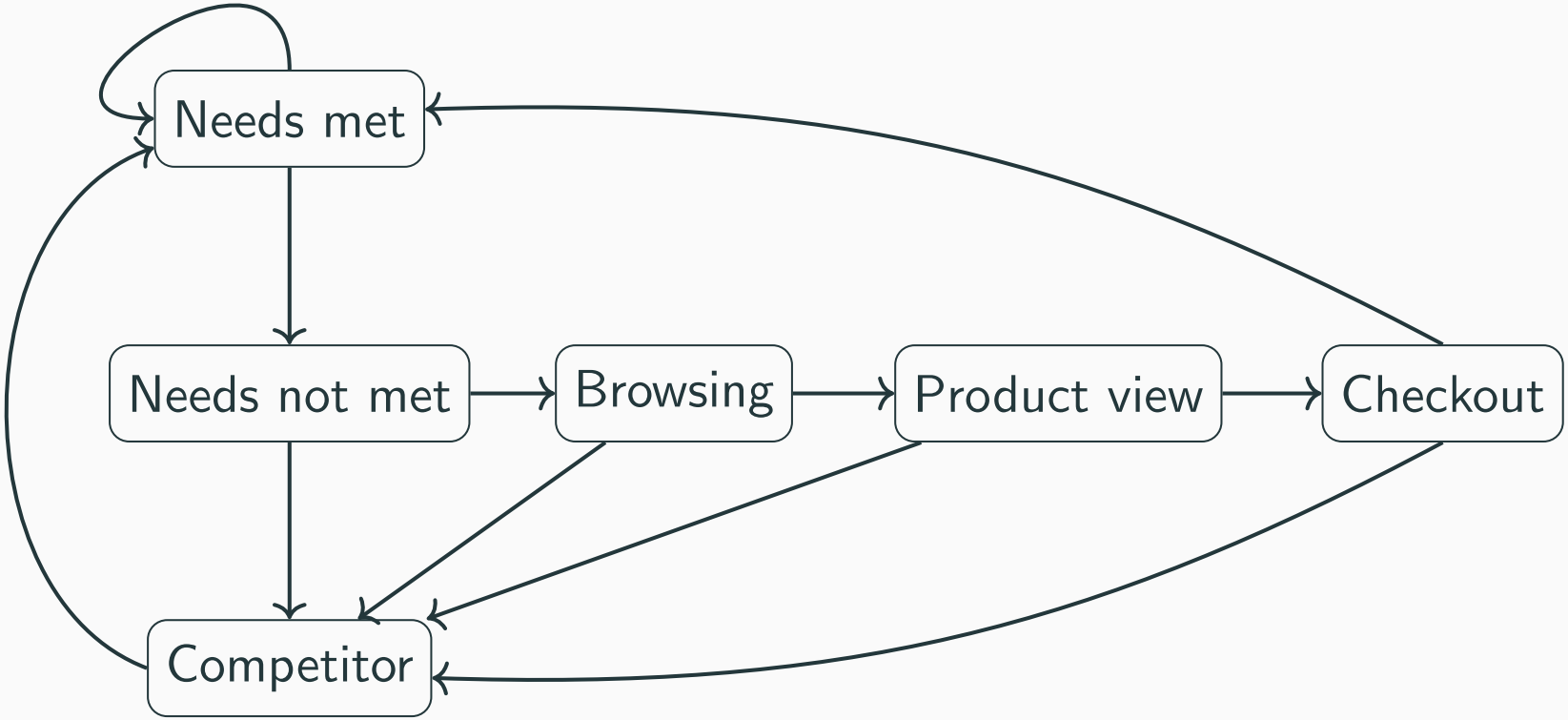
How do you model a probabilistic system?

1. describe states and transitions
2. make number of state finite by abstraction
3. assign probabilities to transitions

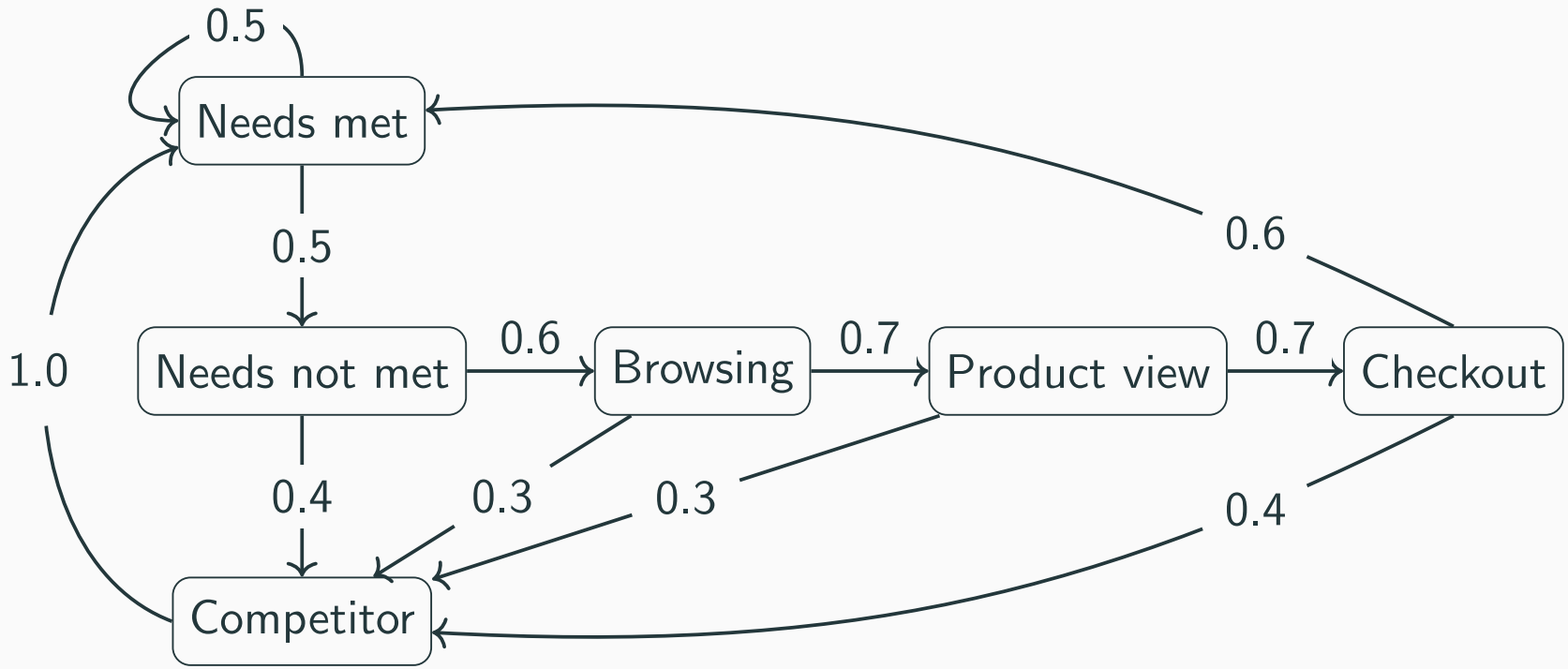
Modeling customers on a shopping website



Modeling customers on a shopping website



Modeling customers on a shopping website



Formalize the model

- **Set of states**

$$S = \{\text{Needs\_met}, \text{Needs\_not\_met}, \text{Competitor}, \ \ldots\}$$

- **Initial state**

$$s_{\text{start}} = \text{Needs\_met}$$

- **Transition probability matrix**

$$P : S \times S \to [0, 1]$$
where $\sum_{s' \in S} P(s, s') = 1$ for all $s \in S$

Formalize the model

$$
\begin{array}{c}
 \\
\text{Needs\_met} \\
\text{Needs\_not\_met} \\
\vdots
\end{array}
\begin{array}{c}
\begin{array}{ccc}
\text{Needs\_met} & \text{Needs\_not\_met} & \cdots
\end{array} \\
\begin{pmatrix}
0.5 & 0.5 & \cdots \\
0 & 0 & \cdots \\
1.0 & 0 & \cdots \\
\vdots & \vdots & \ddots
\end{pmatrix}
\end{array}
$$

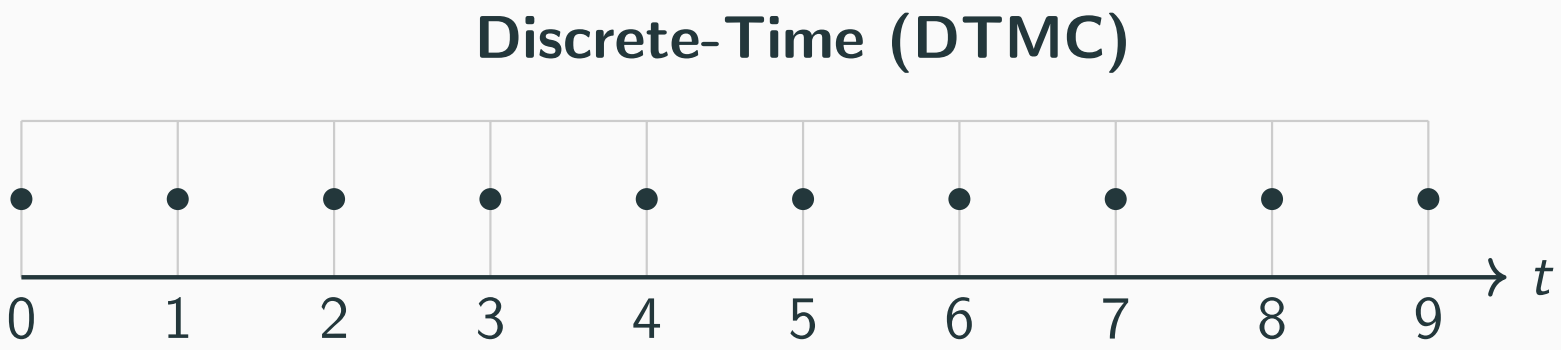
Additional features?

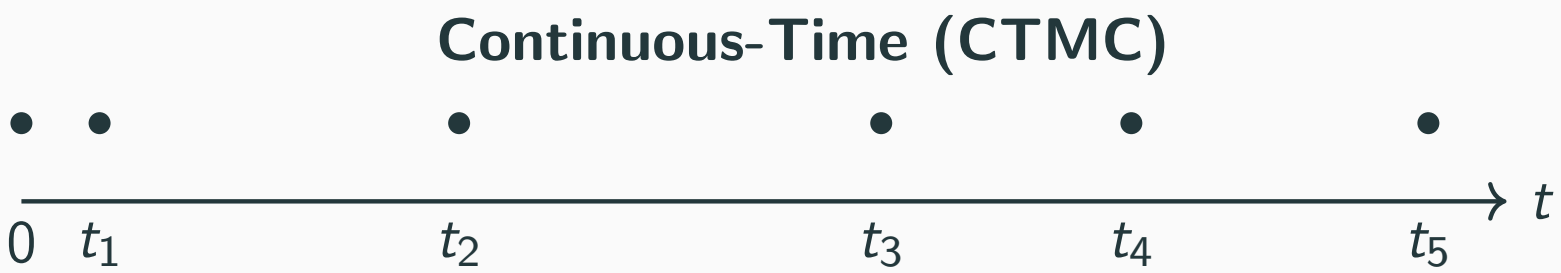
- Discrete vs continuous time
- Nondeterminism
- Rewards
- Intervals
- ...

The Many Kinds of Models

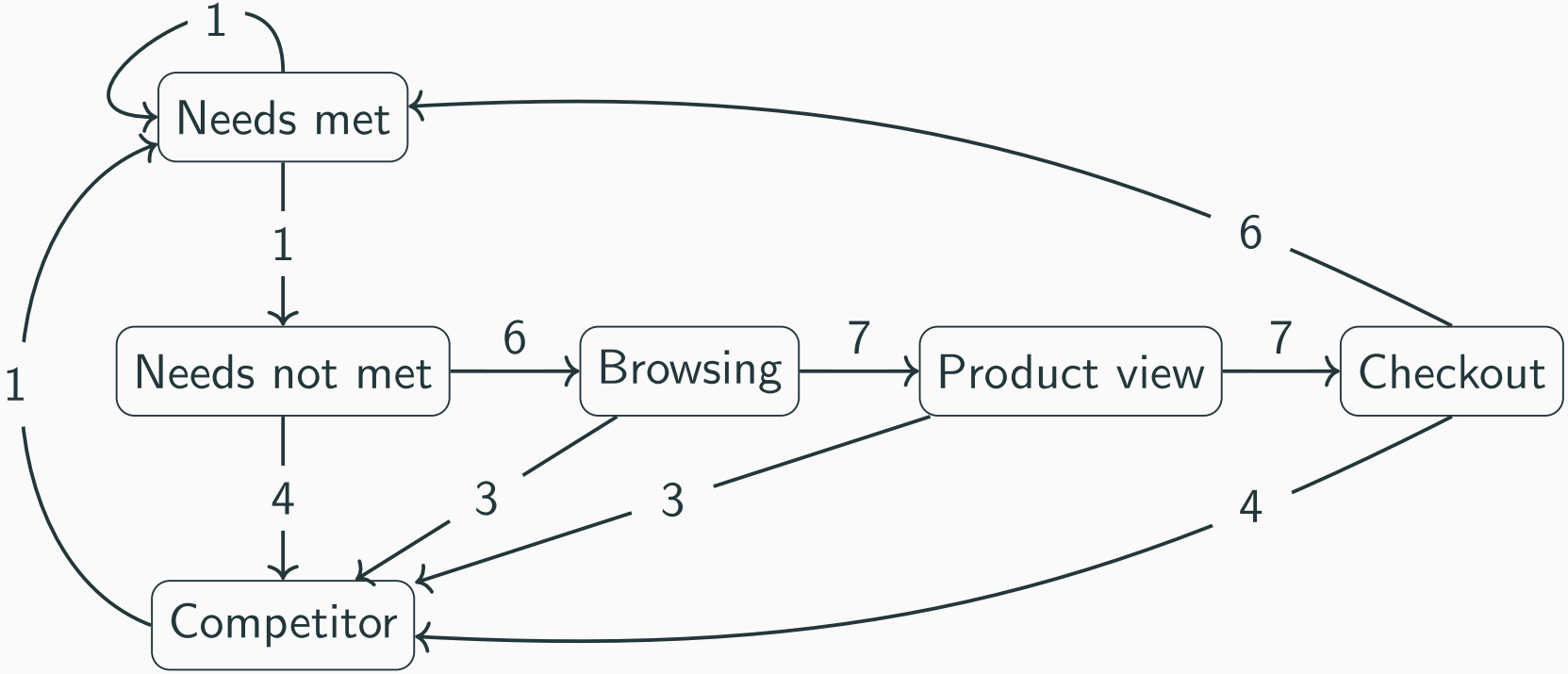
- Discrete-time Markov chains
- Continuous-time Markov chains
- Markov decision processes
- Markov automata
- Probabilistic timed automata
- ...

Continuous vs Discrete time





Continuous-time Markov chains



Continuous-time Markov chains

- **Set of states**

$$S = \{\text{Needs\_met}, \text{Needs\_not\_met}, \text{Competitor}, \ldots\}$$

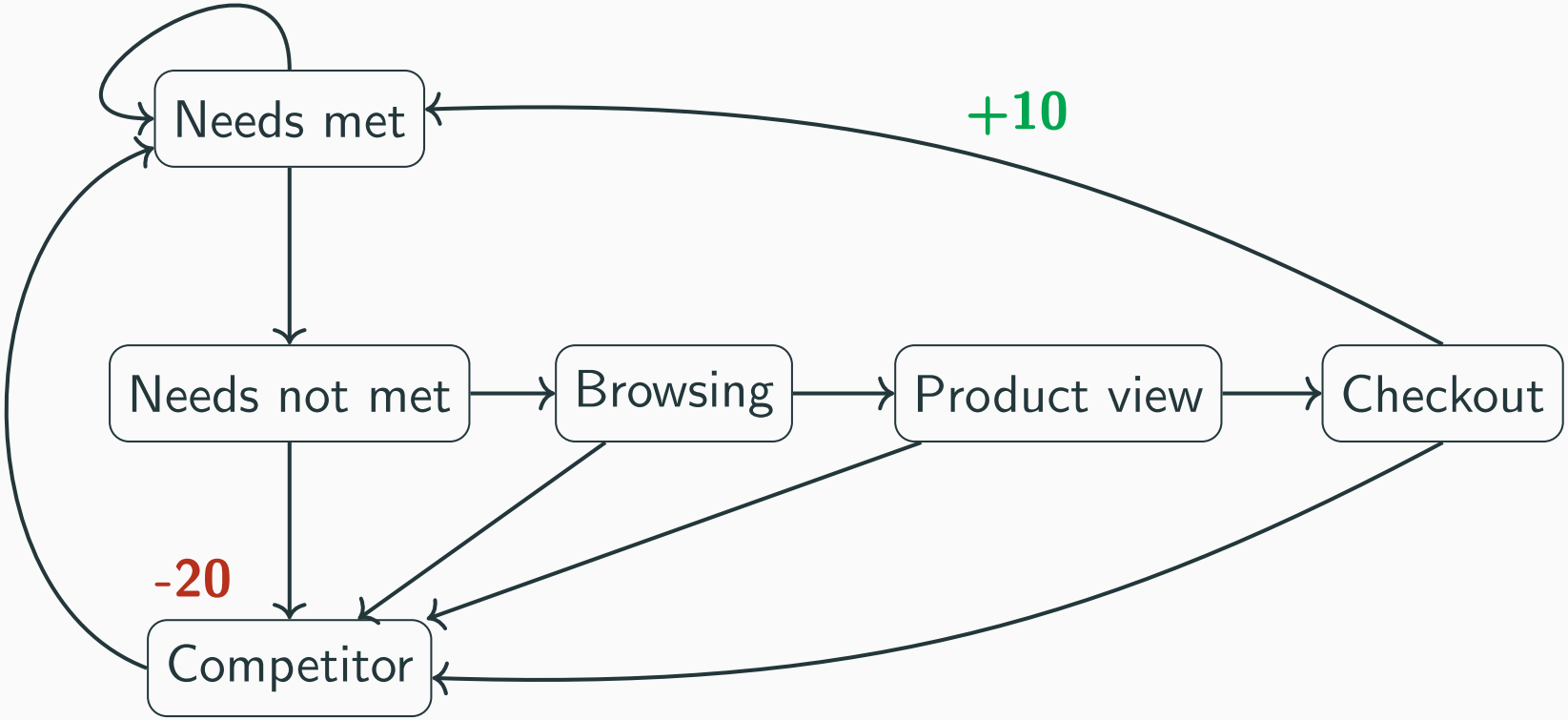
- **Initial state**

$$s_{\text{start}} = \text{Needs\_met}$$

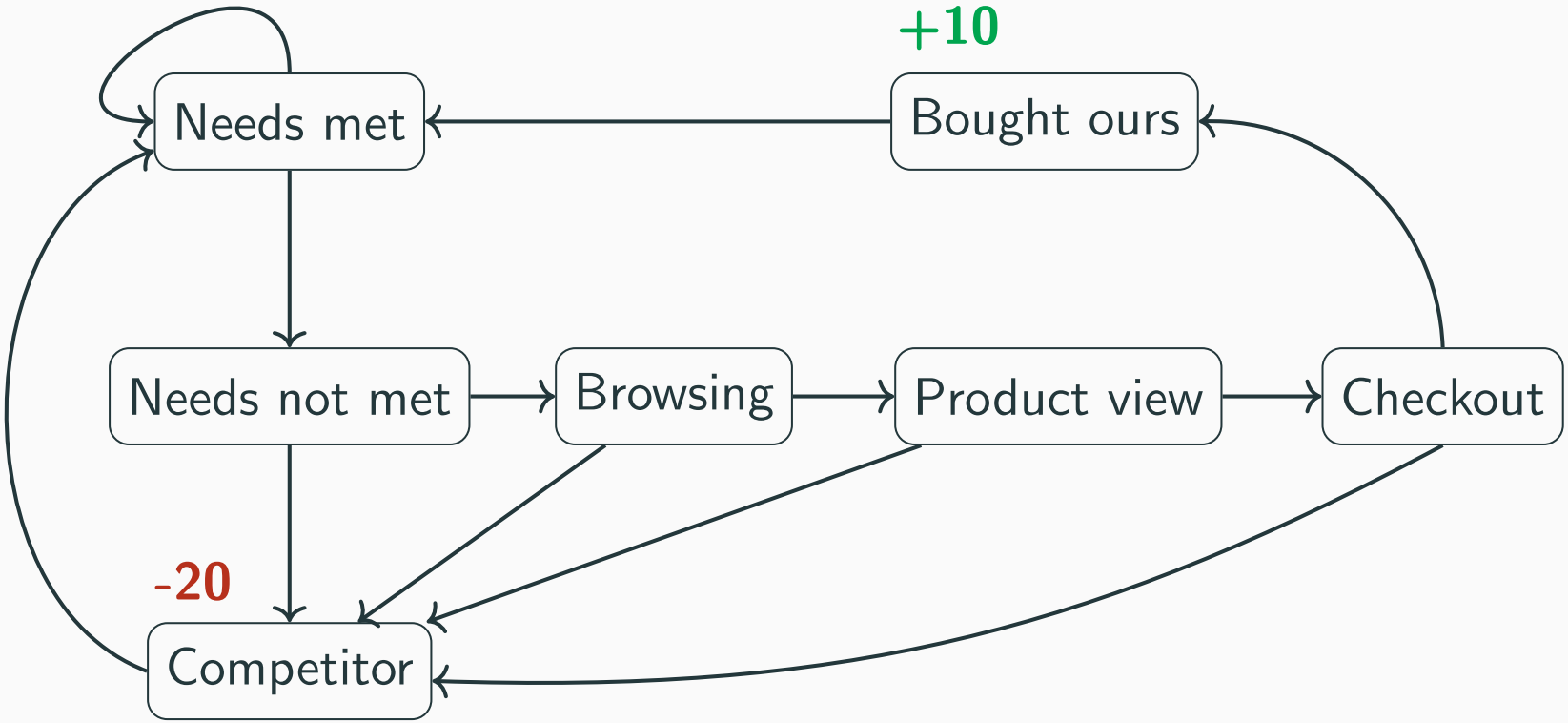
- **Transition rate matrix**

$$Q : S \times S \to [0, \infty)$$

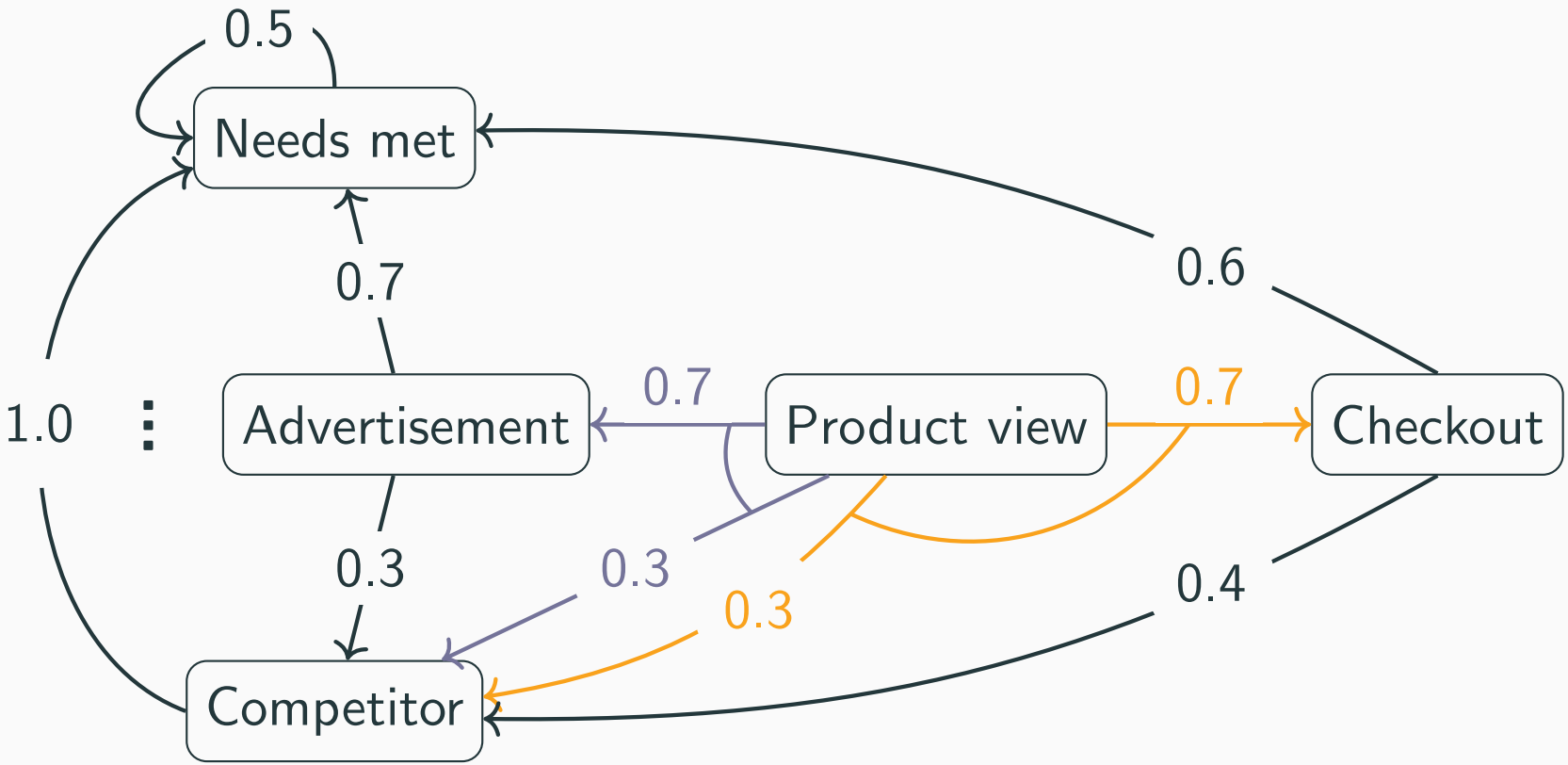
Rewards


Needs met
+10
Needs not met
Browsing
Product view
Checkout
-20
Competitor

Rewards


+10
Needs met
Bought ours
Needs not met
Browsing
Product view
Checkout
-20
Competitor

Nondeterminism



Formalize nondeterminism

- **Set of states**

$$S = \{\text{Needs\_met}, \text{Advertisement}, \text{Competitor}, \ldots\}$$

- **Initial state**

$$s_{\text{start}} = \text{Needs\_met}$$

- **Transition probability function**

$T : S \to 2^{\text{Dist}(S)}$

$$T(\text{Product\_view}) := \{\{\mathit{Advertisement} \mapsto 0.7, \mathit{Competitor} \mapsto 0.3\}, \{\mathit{Checkout} \mapsto 0.7, \mathit{Competitor} \mapsto 0.3\}\}$$

Table of types of models

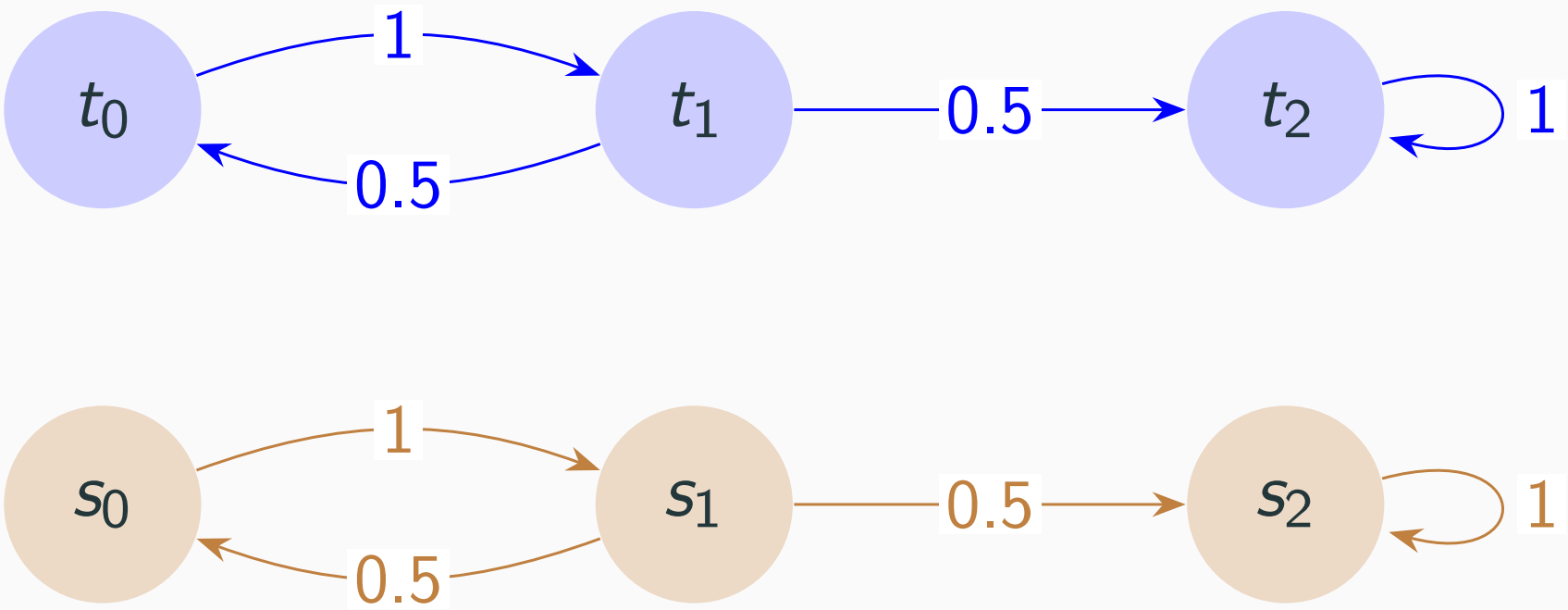
	Discrete time	**Continuous time**
Deterministic	DTMC	CTMC
Nondeterministic	MDP	

Other features

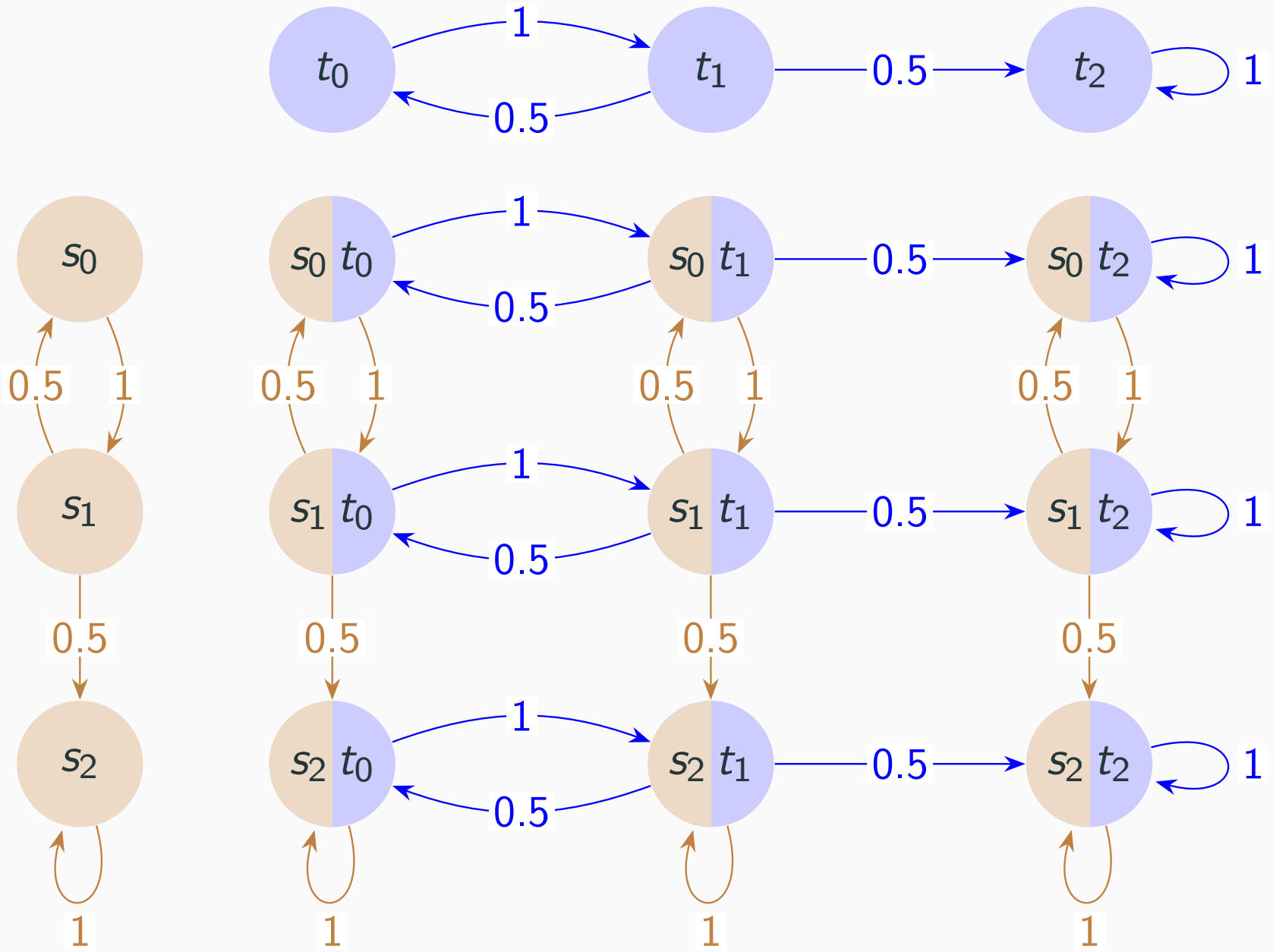
Other features include:

- uncertainty (IMDP, IDTMC)
- timing (PTA)
- partial observability (POMDP, POPTA)
- game-theoretic models (SMG, TSG, CSG, TPTG)

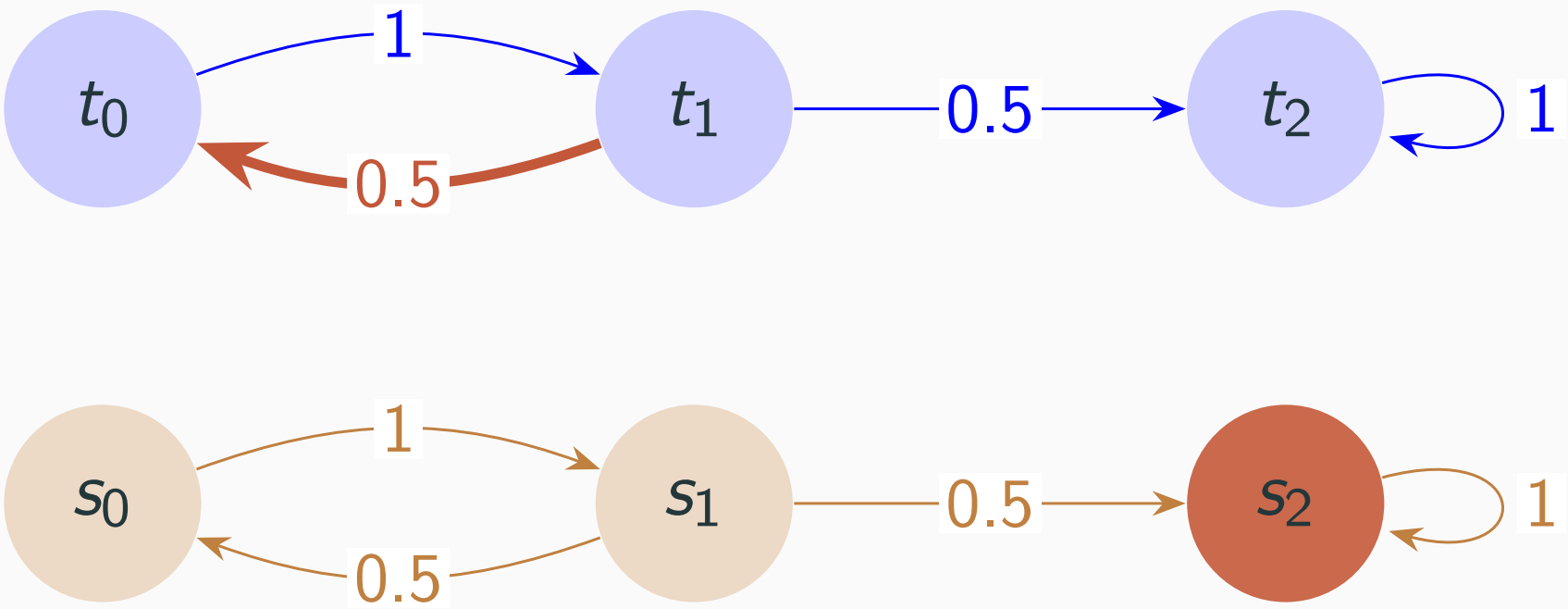
Simple Parallel Composition



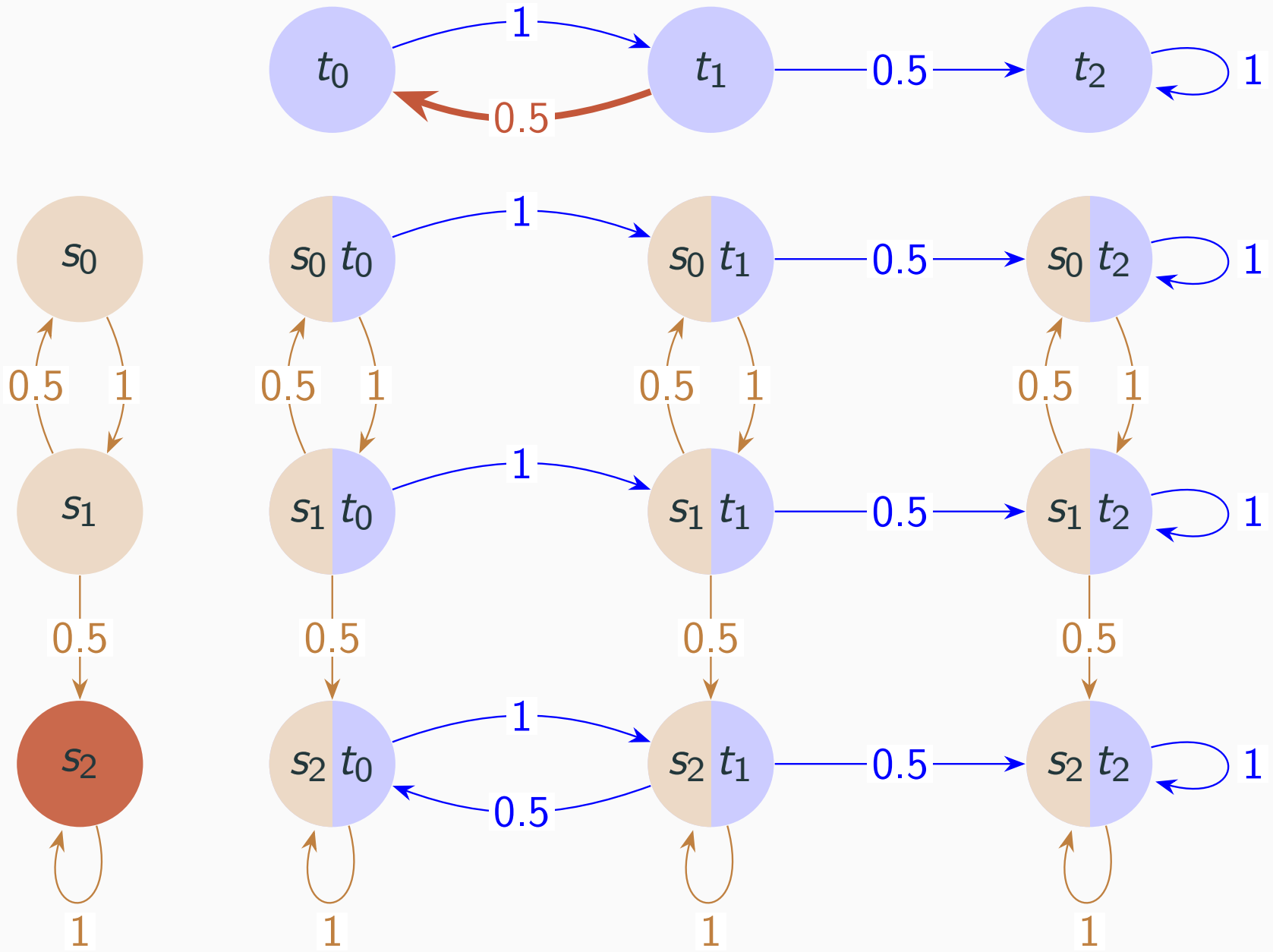
Simple Parallel Composition



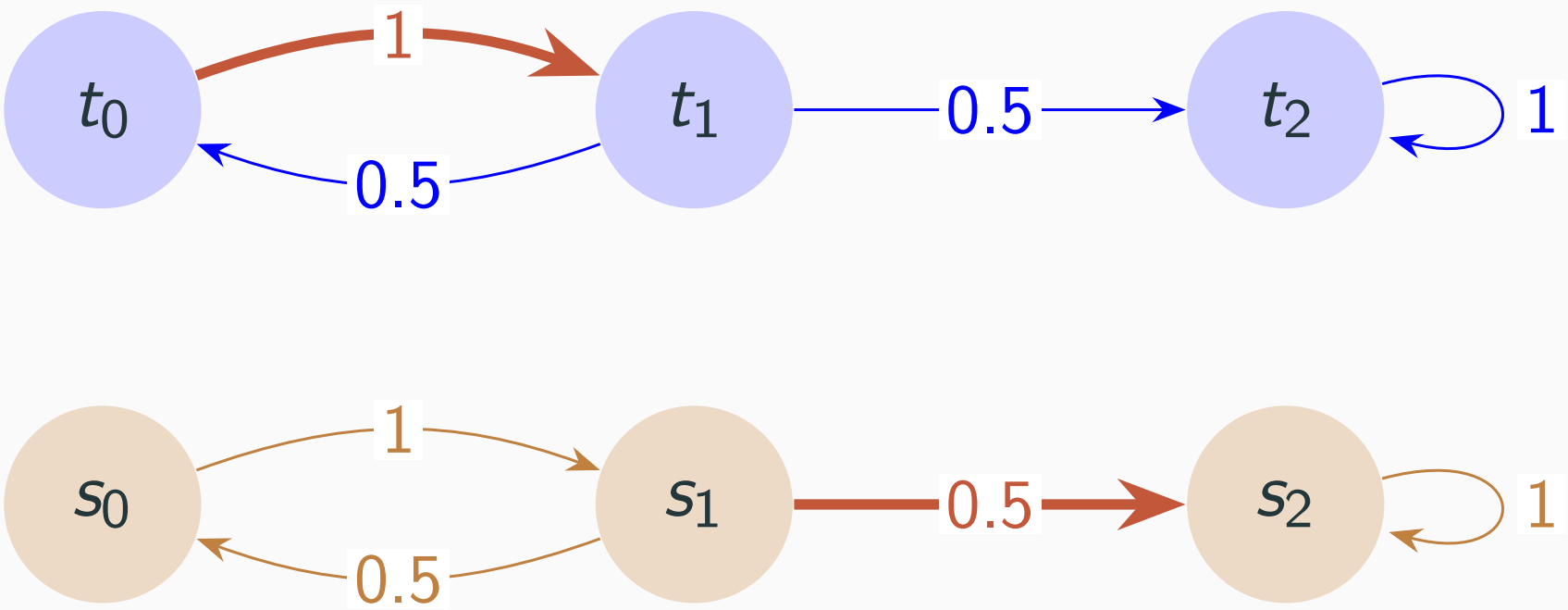
Simple Parallel Composition - node-edge synchronization



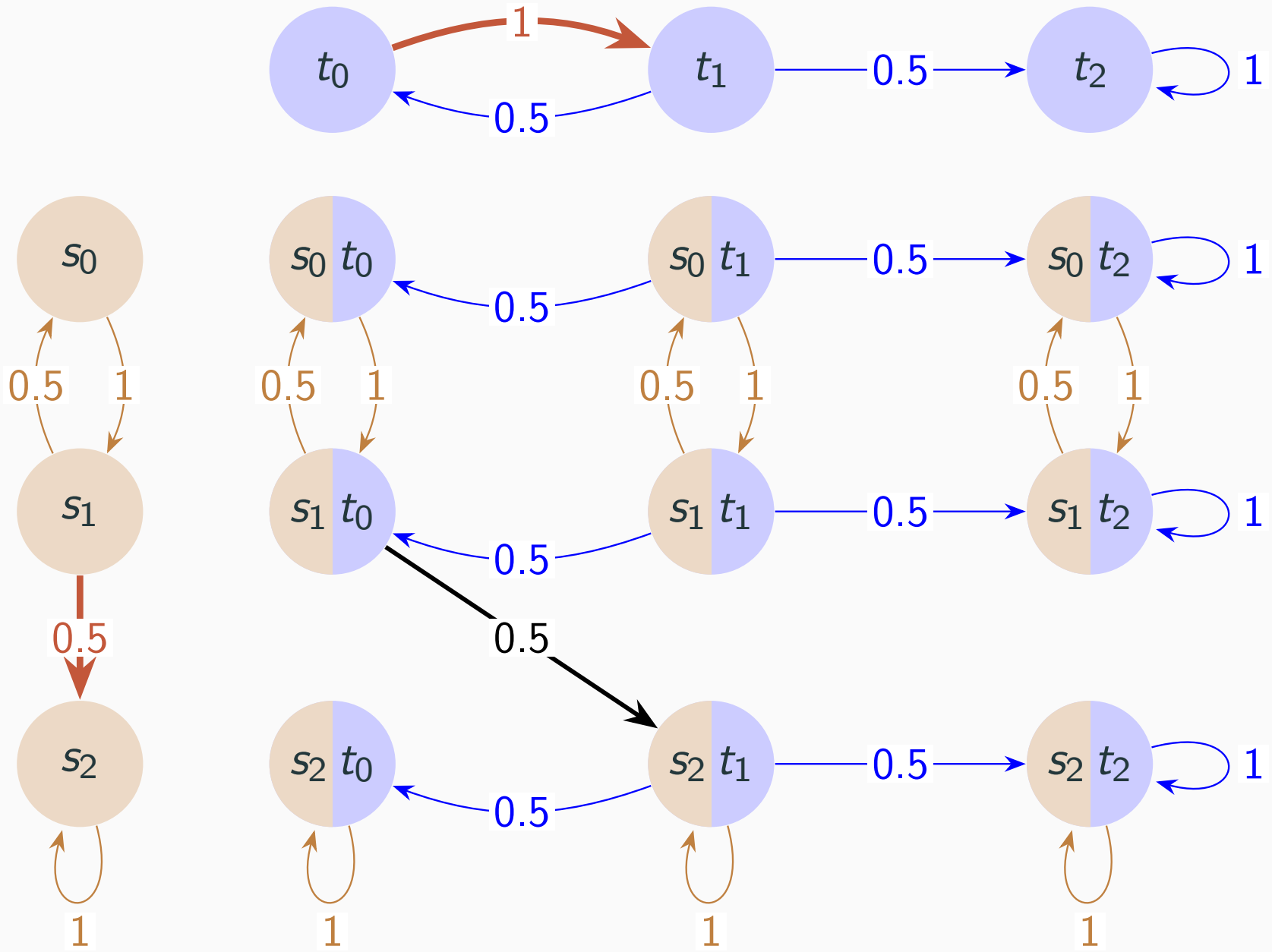
Simple Parallel Composition - node-edge synchronization



Simple Parallel Composition - synchronize on edges



Simple Parallel Composition - synchronize on edges



CTL vs PCTL

CTL - Computation Tree Logic

PCTL - **Probabilistic** Computation Tree Logic

addition: probabilistic operator P

CTL vs PCTL - definitions

- State formulae:

  $\phi ::= \text{true} \mid a \mid \phi \wedge \phi \mid \neg\phi \mid \forall\, \psi \mid \exists, \psi$

  $\phi ::= \text{true} \mid a \mid \phi \wedge \phi \mid \neg\phi \mid P_{\sim p}[\psi]$,

  where a is an atomic proposition, $p \in [0, 1]$ is a probability bound, $\sim \in \{<, >, \leq, \geq\}$, $k \in \mathbb{N}$

- Path formulae:

  $\psi ::= X\, \phi \mid F\, \phi \mid G\, \phi \mid \phi\, U\, \phi$

  where X means "next", G means "globally", and U means "until"

  $\psi ::= X\, \phi \mid \phi\, U^{\leq k}\, \phi \mid \phi\, U\, \phi$,

  where X means "next", U means "until", and $U^{\leq k}$ means "bounded until"

  (path formulae only occur inside the P operator)

CTL vs PCTL - semantics of state formulae

$s \models \phi$ denotes "s satisfies ϕ" or "ϕ is true in s"

For a state s of an LTS $(S, s_{\text{init}}, \rightarrow, L)$:

$$s \models \text{true} \qquad \text{always}$$

$$s \models a \iff a \in L(s)$$

$$s \models \phi_1 \wedge \phi_2 \iff s \models \phi_1 \text{ and } s \models \phi_2$$

$$s \models \neg\phi \iff s \not\models \phi$$

$$s \models \forall\psi \iff \omega \models \psi \text{ for all } \omega \in \mathrm{Path}(s)$$

$$s \models \exists\psi \iff \omega \models \psi \text{ for some } \omega \in \mathrm{Path}(s)$$

(we will mention operators $\forall$ and $\exists$ in PCTL later)

CTL vs PCTL - semantics of path formulae

$\omega \models \psi$ denotes "ω satisfies ψ" or "ψ is true along ω"

For a path ω of an LTS $(S, s_{\text{init}}, \rightarrow, L)$:

$$\omega \models X\,\phi \iff \omega(1) \models \phi$$

$$\omega \models F\,\phi \iff \exists k \geq 0 \text{ s.t. } \omega(k) \models \phi$$

$$\omega \models G\,\phi \iff \forall i \geq 0\ \omega(i) \models \phi$$

$$\omega \models \phi_1\,U\,\phi_2 \iff \exists k \geq 0 \text{ s.t. } \omega(k) \models \phi_2 \text{ and } \forall i < k\ \omega(i) \models \phi_1$$

$$\omega \models \phi_1\,U^{\leq k}\,\phi_2 \iff \exists i \leq k \text{ such that } \omega(i) \models \phi_2 \text{ and } j < i,\ \omega(j) \models \phi_1$$

PCTL - semantics of the probability operator

Notation:

$$s \models P_{\sim p}[\psi]$$

means that the probability, from state s, that ψ is true for an outgoing path satisfies $\sim p$.

Example:

$$s \models P_{<0.15}[X \text{ succ}]$$

$\iff$ "the probability of succeed in the next state of outgoing paths from s is less than 0.15"

formally:

$$s \models P_{\sim p}[\psi] \iff \mathrm{Prob}(s, \psi) \sim p$$

where:

$$\mathrm{Prob}(s, \psi) = \Pr_{s} \{\omega \in \mathrm{Path}(s) \mid \omega \models \psi\}$$

PCTL - basic logical equivalences

False:

$$\text{false} \equiv \neg\text{true}$$

Disjunction:

$$\phi_1 \vee \phi_2 \equiv \neg(\neg\phi_1 \wedge \neg\phi_2)$$

Implication:

$$\phi_1 \rightarrow \phi_2 \equiv \neg\phi_1 \vee \phi_2$$

Negation example:

$$\neg P_{>p}[\phi_1 \ U \ \phi_2] \equiv P_{\leq p}[\phi_1 \ U \ \phi_2]$$

PCTL - examples

- "The probability that the sender has received n acknowledgements within k clock–ticks is at least 0.8":

$$P_{\geq 0.8}\left[F^{\leq k}\ \texttt{reply\_count} = n\right]$$

- "The probability that component B fails before component A is less than 0.4"

$$P_{<0.4}\left[\neg \texttt{fail}_A\ U\ \texttt{fail}_B\right]$$

PCTL - operators $\forall$ and $\exists$

We use P operator to define $\forall$ and $\exists$.

- **Qualitative** PCTL properties

  $P_{\sim p}[\psi]$, where p is either 0 or 1.

- **Quantitative** PCTL properties

  $P_{\sim p}[\psi]$, where p is in the range $(0, 1)$.

$\Longrightarrow$

$P_{>0}[F\ \phi]$ is identical to $\exists F\ \phi$

there exists a finite path to a ϕ–state

$P_{\geq 1}[F\ \phi]$ is (similar to but) weaker than $\forall F\ \phi$

a ϕ–state is reached "almost surely"

PCTL - limitations

PCTL is very useful for specifying **probabilistic reachability** properties

- e.g. "what is the probability of eventually reaching a failure/success state?"
- most PCTL properties are variations of:

$$\phi \; U \; \psi$$

meaning: "stay in ϕ states until reaching a ψ state"

Therefore, PCTL mainly reasons about:

- reaching target states
- staying within allowed states beforehand
- probabilities of these events
- possibly within bounded time

PCTL - limitations

However, PCTL has limited expressivity

- difficult to express complex long–term behaviours
- e.g. “every request is eventually acknowledged”
- e.g. “event A happens infinitely often”

More expressive temporal logics include:

- PCTL*, which combines probabilities with richer temporal reasoning

How does PCTL model checking work?

Given a DTMC $\mathcal{D} = (S, s_{\text{init}}, \mathbf{P}, L)$ and a PCTL formula ϕ:

Goal: compute the *satisfaction set* $\mathrm{Sat}(\phi) = \{s \in S \mid s \models \phi\}$

Algorithm: bottom-up on the syntax tree of ϕ

1. Parse ϕ into subformulae
2. Process from leaves to root:
   - Atomic props, $\neg$, $\wedge$: same as CTL (set operations)
   - $P_{\sim p}[X\, \phi_1]$: **matrix–vector multiplication**
   - $P_{\sim p}[\phi_1\, U^{\leq k}\, \phi_2]$: **iterative** matrix–vector multiplication
   - $P_{\sim p}[\phi_1\, U\, \phi_2]$: **linear equation system**
3. Check whether $s_{\text{init}} \in \mathrm{Sat}(\phi)$

[Baier & Katoen, Ch. 10; Parker, Lecture 5]

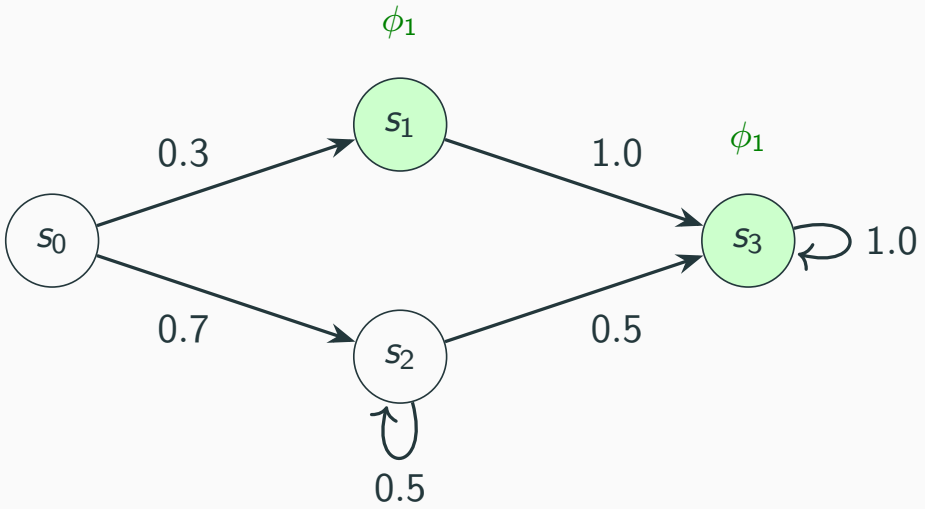
PCTL model checking — Next operator

Goal: check $P_{\sim p}[X\,\phi_1]$ for each state s.

For each state s, compute the probability of reaching a ϕ_1–state in one step:

$$\mathrm{Prob}(s, X\,\phi_1) \;=\; \sum_{s' \in \mathrm{Sat}(\phi_1)} \mathbf{P}(s, s')$$

This is a single **matrix–vector product**: $\vec{x} = \mathbf{P} \cdot \vec{v}$, where $\vec{v}$ is the characteristic vector of $\mathrm{Sat}(\phi_1)$.



$\vec{v} = (0, 1, 0, 1)^T$

$\vec{x} = \mathbf{P}\,\vec{v}$

$x_{s_0} = 0.3 \cdot 1 + 0.7 \cdot 0 = 0.3$

$x_{s_2} = 0.5 \cdot 0 + 0.5 \cdot 1 = 0.5$

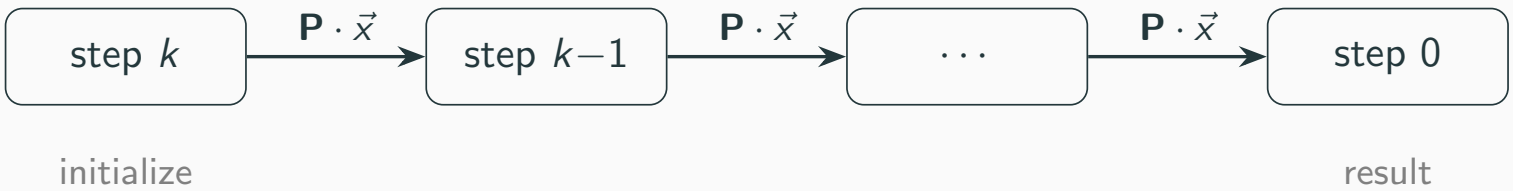
Then: $s \in \mathrm{Sat}(P_{\sim p}[X\,\phi_1])$ iff $x_s \sim p$.

PCTL model checking — Bounded until

Goal: check $P_{\sim p}[\phi_1 \, U^{\leq k} \, \phi_2]$.

Compute probability by **dynamic programming** (backward from step k):

$$x_s^{(t)} = \begin{cases} 1 & \text{if } s \models \phi_2 \\ 0 & \text{if } s \not\models \phi_1 \text{ or } t = 0 \\ \displaystyle\sum_{s' \in S} \mathbf{P}(s, s') \cdot x_{s'}^{(t-1)} & \text{otherwise} \end{cases}$$



k iterations of matrix–vector multiplication.

Complexity: $\mathcal{O}(k \cdot |S|^2)$ (or $\mathcal{O}(k \cdot |\text{transitions}|)$ with sparse matrices).

PCTL model checking — Unbounded until (the hard case)

Goal: check $P_{\sim p}[\phi_1 \, U \, \phi_2]$.

Step 1: Precomputation (graph analysis, same as CTL):

- $S^{\text{no}} = \{s \mid \text{Prob}(s, \phi_1 U \phi_2) = 0\}$ — ϕ_2 unreachable via ϕ_1–states
- $S^{\text{yes}} = \{s \mid \text{Prob}(s, \phi_1 U \phi_2) = 1\}$ — reach ϕ_2 almost surely

Both found by **backward graph search** (BFS/DFS).

Step 2: Solve a linear equation system for the remaining states $S^{?} = S \setminus (S^{\text{no}} \cup S^{\text{yes}})$:

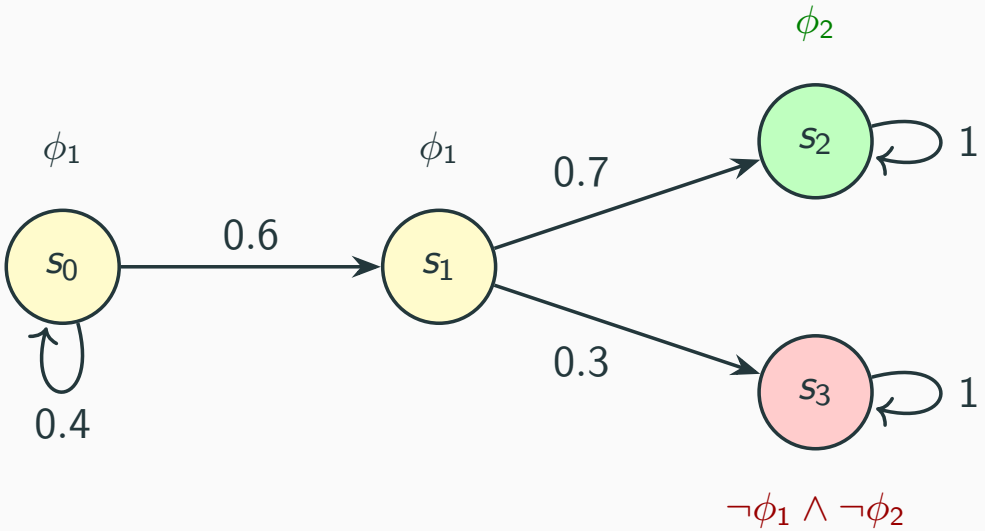
$$x_s = \sum_{s' \in S} \mathbf{P}(s, s') \cdot x_{s'} \qquad \text{with } x_s = 0 \text{ for } s \in S^{\text{no}}, \;\; x_s = 1 \text{ for } s \in S^{\text{yes}}$$

Step 3: Check $x_s \sim p$ for each state s.

Solving methods: Gaussian elimination (exact) or iterative (Jacobi, Gauss–Seidel — used in PRISM).

Unbounded until — visual example

Check: $P_{\geq 0.5}[\phi_1 \, U \, \phi_2]$



$S^{\text{yes}} = \{s_2\}$ $\quad x_{s_2} = 1$

$S^{\text{no}} = \{s_3\}$ $\quad x_{s_3} = 0$

$S^{?} = \{s_0, s_1\}$:

$x_{s_1} = 0.7 \cdot 1 + 0.3 \cdot 0 = 0.7$

$x_{s_0} = 0.6 \cdot x_{s_1} + 0.4 \cdot x_{s_0}$

$\implies x_{s_0} = 0.7$

$0.7 \geq 0.5$ ✓

PCTL model checking — CTL vs PCTL comparison

Aspect	CTL	PCTL
Path quantifiers	$\forall$, $\exists$	$P_{\sim p}$
Core computation	graph fixpoint (BFS/DFS)	numerical (matrix ops + linear systems)
Model	Kripke structure	DTMC (or MDP)
Result per state	Boolean	probability, then compare against p
Complexity:		
CTL	$\mathcal{O}(\lvert\phi\rvert \cdot (\lvert S\rvert + \lvert R\rvert))$ — graph traversal	
PCTL	$\mathcal{O}(\lvert\phi\rvert \cdot (\text{poly}(\lvert S\rvert) + \text{equation solving}))$	

The shift can be summarized:
CTL asks "does this hold on all/some paths?"
PCTL asks "does the **probability** of this holding meet a threshold?"

[Baier & Katoen, *Principles of Model Checking*, 2008]

Extension to MDPs

For **Markov Decision Processes** (nondeterminism + probability):

- Same PCTL syntax, but semantics must account for **all possible schedulers** (adversaries)
- Model checking computes **min/max probabilities**:

$$\Pr_s^{\min}(\psi) \quad \text{and} \quad \Pr_s^{\max}(\psi)$$

- For unbounded until: solve a **linear program** (LP) instead of a linear equation system
- Key result: *memoryless deterministic* schedulers suffice for reachability properties

[Parker, Lecture 14; Baier & Katoen, Ch. 10.6]

PRISM: The Foundational Model Checker

PRISM model checker, primarily developed at the University of Oxford, is the most widely used tool in the field that supports various types of models, including:

- Discrete-Time Markov Chains (DTMCs)
- Markov Decision Processes (MDPs)
- Continuous-Time Markov Chains (CTMCs)
- Probabilistic Timed Automata (PTAs)

Its primary strength lies in its use of symbolic data structures, specifically Binary Decision Diagrams (BDDs) and Multi-Terminal Binary Decision Diagrams (MTBDDs). These structures allow PRISM to represent and analyze models with billions of states by exploiting the structural regularity of the system.

PRISM includes several analysis engines:

- Symbolic Engine: Uses MTBDDs for both representation and numerical solution.
- Explicit Engine: Uses sparse matrices and is often faster for models with less regularity.
- Hybrid Engine: Combines the memory efficiency of MTBDDs with the speed of sparse matrices.
- Simulator: Supports Statistical Model Checking (SMC), which uses Monte Carlo simulation to estimate probabilities when the state space is too large for exhaustive analysis.

PRISM-games extends its capabilities to stochastic multi-player games (systems with multiple agents)

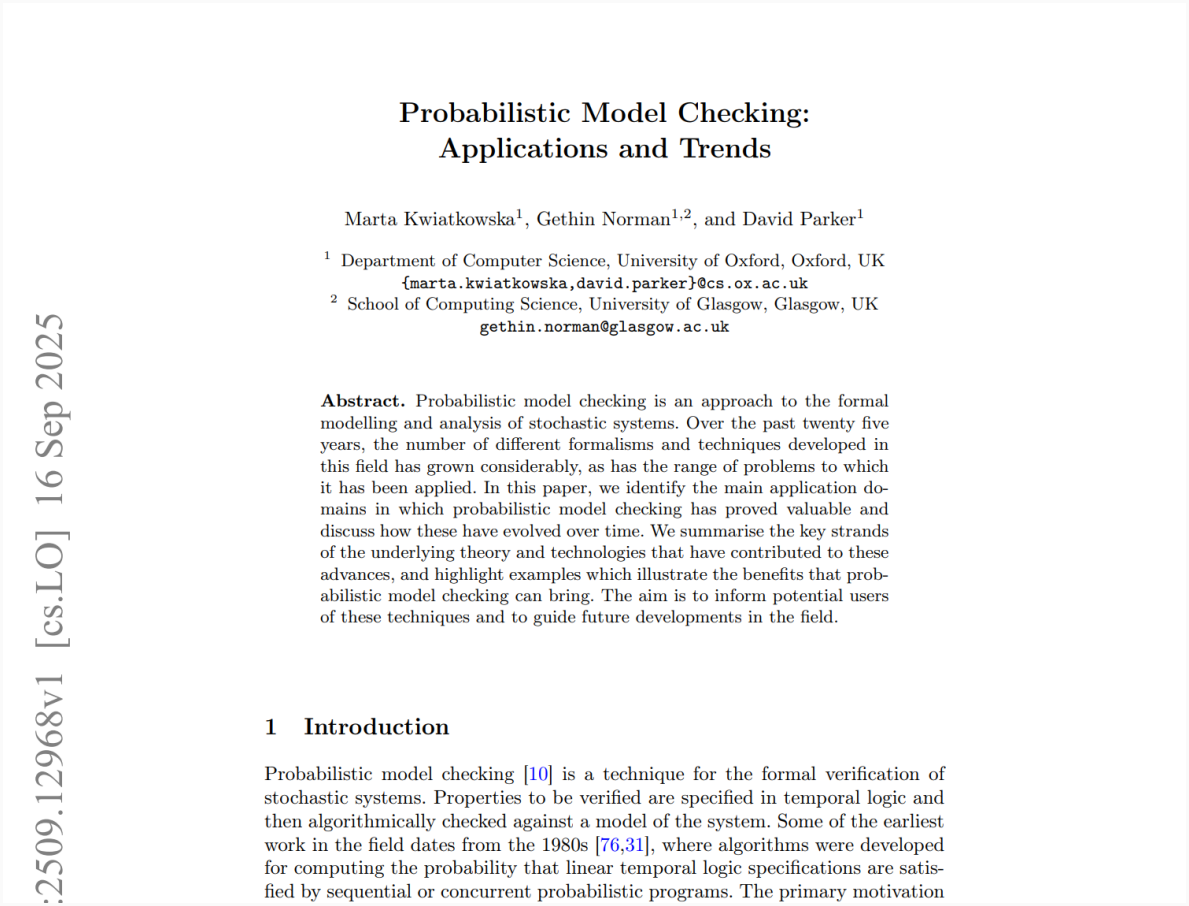
Diverse Application Domains

The versatility of probabilistic model checking has led to its adoption across a wide range of application domains, including:

- Randomised distributed algorithms
- Communication protocols and networks
- Computer security
- Autonomous systems and robotics
- Safety-critical systems
- Biology and human behavior
- AI and self-adaptive systems
- Many more...

In the next few slides, we will briefly explore some of these application areas and the types of questions that probabilistic model checking can help answer. So many applications that we cannot cover them all, but we will give you a flavour of the diversity of applications and the types of questions that can be answered.

Reference Paper



Probabilistic Model Checking: Applications and Trends

Marta Kwiatkowska[1], Gethin Norman[1,2], and David Parker[1]

[1] Department of Computer Science, University of Oxford, Oxford, UK
{marta.kwiatkowska,david.parker}@cs.ox.ac.uk
[2] School of Computing Science, University of Glasgow, Glasgow, UK
gethin.norman@glasgow.ac.uk

Abstract. Probabilistic model checking is an approach to the formal modelling and analysis of stochastic systems. Over the past twenty five years, the number of different formalisms and techniques developed in this field has grown considerably, as has the range of problems to which it has been applied. In this paper, we identify the main application domains in which probabilistic model checking has proved valuable and discuss how these have evolved over time. We summarise the key strands of the underlying theory and technologies that have contributed to these advances, and highlight examples which illustrate the benefits that probabilistic model checking can bring. The aim is to inform potential users of these techniques and to guide future developments in the field.

1 Introduction

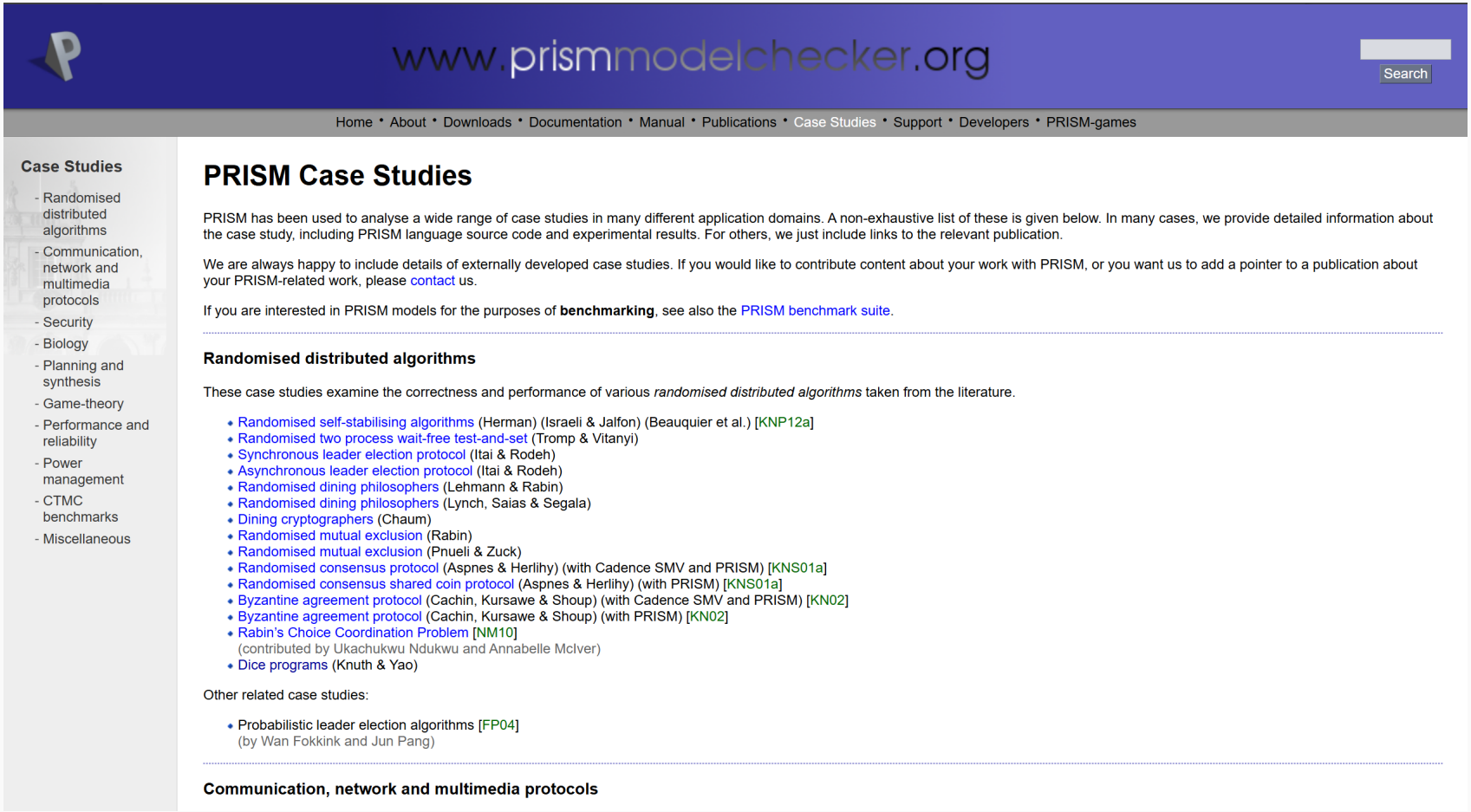
Probabilistic model checking [10] is a technique for the formal verification of stochastic systems. Properties to be verified are specified in temporal logic and then algorithmically checked against a model of the system. Some of the earliest work in the field dates from the 1980s [76,31], where algorithms were developed for computing the probability that linear temporal logic specifications are satisfied by sequential or concurrent probabilistic programs. The primary motivation

Probabilistic Model Checking: Applications and Trends

Kwiatkowska et al., 2025

https://arxiv.org/abs/2509.12968

PRISM Case Studies



www.prismmodelchecker.org

Search

Home • About • Downloads • Documentation • Manual • Publications • Case Studies • Support • Developers • PRISM-games

Case Studies

- Randomised distributed algorithms
- Communication, network and multimedia protocols
- Security
- Biology
- Planning and synthesis
- Game-theory
- Performance and reliability
- Power management
- CTMC benchmarks
- Miscellaneous

PRISM Case Studies

PRISM has been used to analyse a wide range of case studies in many different application domains. A non-exhaustive list of these is given below. In many cases, we provide detailed information about the case study, including PRISM language source code and experimental results. For others, we just include links to the relevant publication.

We are always happy to include details of externally developed case studies. If you would like to contribute content about your work with PRISM, or you want us to add a pointer to a publication about your PRISM-related work, please contact us.

If you are interested in PRISM models for the purposes of **benchmarking**, see also the PRISM benchmark suite.

Randomised distributed algorithms

These case studies examine the correctness and performance of various *randomised distributed algorithms* taken from the literature.

- Randomised self-stabilising algorithms (Herman) (Israeli & Jalfon) (Beauquier et al.) [KNP12a]
- Randomised two process wait-free test-and-set (Tromp & Vitanyi)
- Synchronous leader election protocol (Itai & Rodeh)
- Asynchronous leader election protocol (Itai & Rodeh)
- Randomised dining philosophers (Lehmann & Rabin)
- Randomised dining philosophers (Lynch, Saias & Segala)
- Dining cryptographers (Chaum)
- Randomised mutual exclusion (Rabin)
- Randomised mutual exclusion (Pnueli & Zuck)
- Randomised consensus protocol (Aspnes & Herlihy) (with Cadence SMV and PRISM) [KNS01a]
- Randomised consensus shared coin protocol (Aspnes & Herlihy) (with PRISM) [KNS01a]
- Byzantine agreement protocol (Cachin, Kursawe & Shoup) (with Cadence SMV and PRISM) [KN02]
- Byzantine agreement protocol (Cachin, Kursawe & Shoup) (with PRISM) [KN02]
- Rabin's Choice Coordination Problem [NM10]
  (contributed by Ukachukwu Ndukwu and Annabelle McIver)
- Dice programs (Knuth & Yao)

Other related case studies:

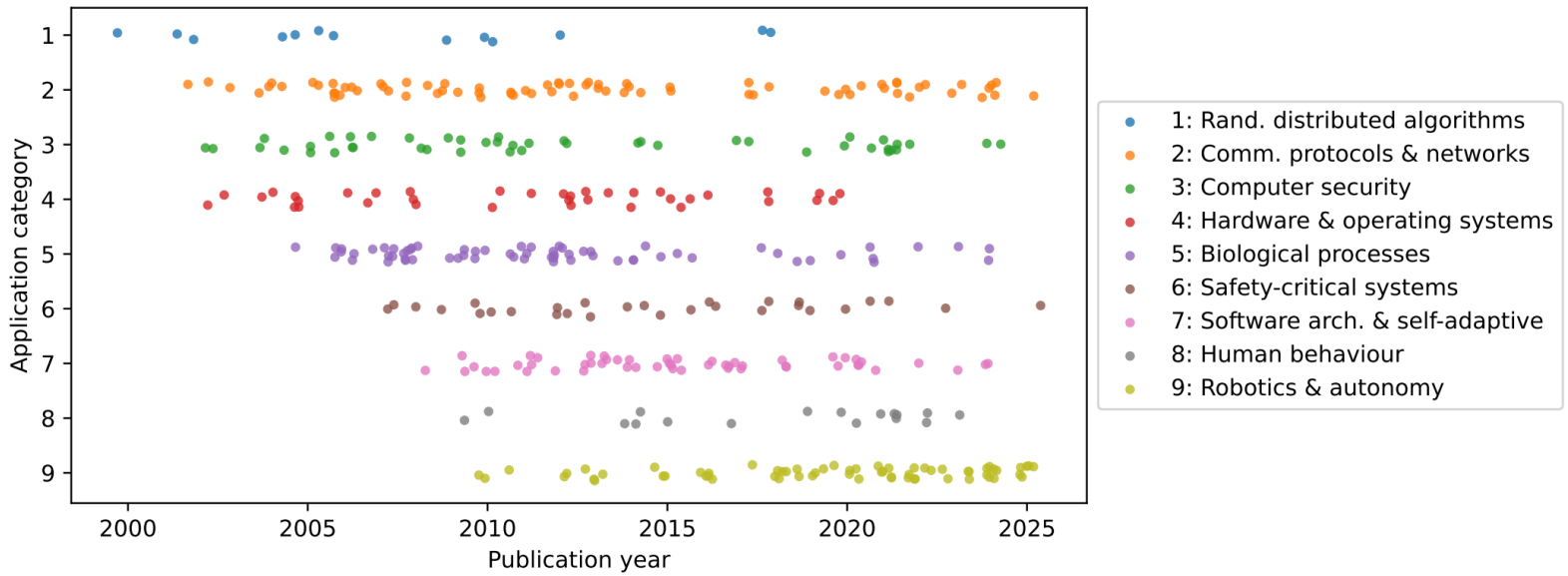
- Probabilistic leader election algorithms [FP04]
  (by Wan Fokkink and Jun Pang)

Communication, network and multimedia protocols

PRISM Case Studies `https://www.prismmodelchecker.org/casestudies/index.php`

Recommended for further reading if you are interested in more details and references to specific papers in each area.

Categories of Application-Oriented Publications



Source: Kwiatkowska et al., 2025

Randomised Distributed Algorithms

- A fruitful initial area of study for probabilistic model checking

Problem: Symmetry breaking between concurrent processes - establishing correct termination and fairness

- Leader election
- Byzantine agreement
- Self-stabilization
- Consensus protocols
- Fault tolerance

Generally uses MDPs since they are ideally suited to model the mixture of stochasticity and concurrency, sometimes DTMCs are also used.

Scalability of verification techniques was enhanced with the development of symbolic methods.

Communication Protocols

- A natural progression from work on distributed systems
- Often deploy randomness to break symmetry
- In wireless we also model underlying medium's unreliability (message delays or losses)

Important examples:

- Blueooth device discovery (multiple DTMC models with $\geq 10^{10}$ states)
- CSMA/CA (Carrier Sense Multiple Access with Collision Avoidance)
- FireWire and Zigbee protocols (use of PTAs)

Modern exaples include wireless protocols , vehicular networks and wireless sensor networks

Security Protocols

Key ideas:

- Attackers are unpredictable (randomized or adaptive attacks)
- Use of randomisation is widespread in security protocols (e.g. key generation)
- Need for strong guarantees (e.g. probability of attack success must be very low)

Natural use of MDPs to model the interaction between the system and an adversary. $\Rightarrow$ **Generalized by stochastic games**

Trade-off between security and performance (e.g. more randomisation can increase security but also increase latency) $\Rightarrow$ The expressivity of probabilistic temporal logics allows us to capture and analyze these trade-offs (e.g. relation between error and transmission cost).

Biology

CTMCs are useful for modeling stochastic dynamics of biological processes:

- Cellular processes (protein to protein interactions, gene expression, chemical reactions)
- Population dynamics (epidemic disease spreading, population growth)

Questions like: *"what is the expected number of molecules of protein X after 5 minutes?" or "what is the probability that more than 10% of the population is infected within 2 weeks?"*

- Various efficient CTMC solution methods but importantly this area has driven the development of *statistical model checking*.
- Can be used to validate hypothesised models and can support *synthetic biology* (e.g. DNA computing)

Safety-Critical Systems

Used to rigorously quantify the likelihood with which high-risk failures or malfunctions could occur

- Assessment of safety integrity levels (SILs), which specify acceptable probabilities or rates of failure
- Modelled as DTMCs or CTMSCs (for quantifying failure rates over time)
- Safety specifications typically amount to relatively simple PCTL or CSL queries

Early exmaple: integration into the failure mode and effects analysis of an airbag system. Also used in:

- Railway infrastructure (using *Fault Tree Analysis*)
- Vehicle guidance systems
- Medical (pacemaker design) and aerospace domain (satellite systems and spacecraft design)

Autonomous Systems & Robotics

- Clear trend in recent years
- Robots operate in uncertain environments (also interaction with other agents) with imperfect sensors and actuators
- Guarantees of policies (safety and reliability) are crucial

Common use of LTL to capture complex temporal specifications like: *"maximise the probability of inspecting all three sensors, in any order, whilst avoiding X"*.
Often combined with **multi-objective model checking** $\Rightarrow$ allows generation of policies that trade-off between several, competing objectives (e.g., battery life vs. mission execution time) or analysis of the corresponding Pareto front.

- Use in AUVs, UAVs, self-driving cars, multi-robot systems
- Highlights challenges

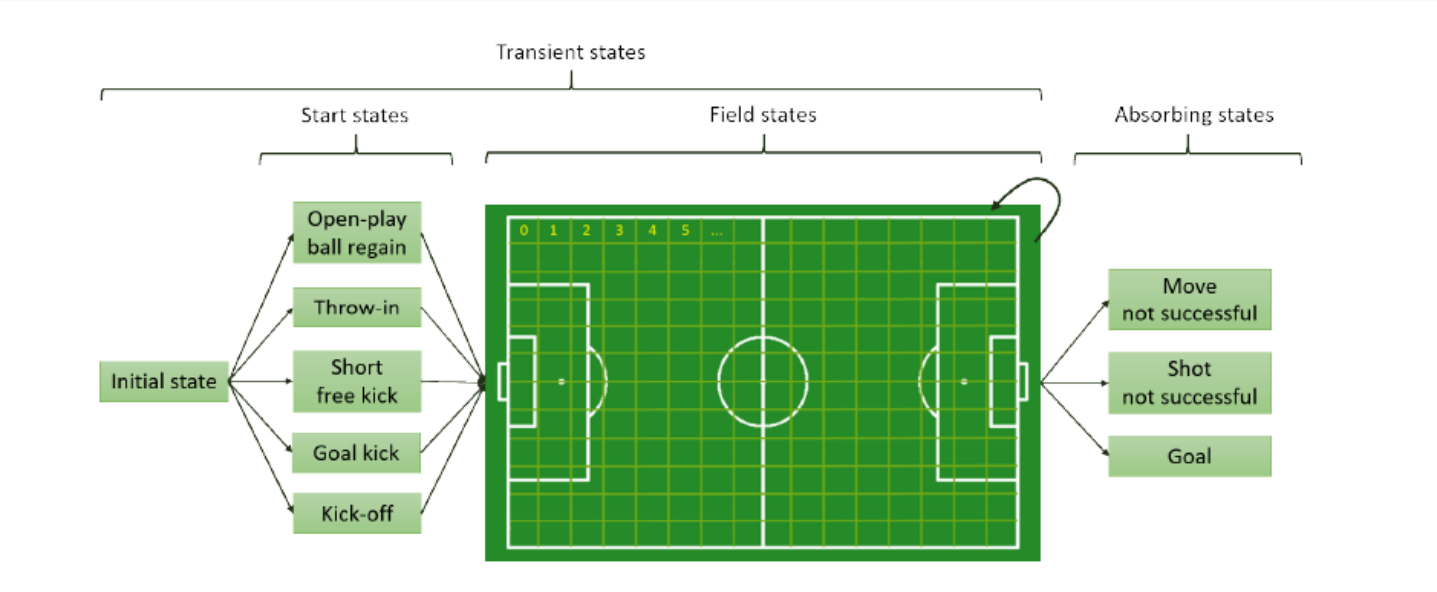
Some further application areas

- Human behaviour modeling
- Software architectures and self-adaptive systems
- Smart Grids (performance, resilience and security)
- Quantum computing (key distribution algorithms)
- Blockchain and cryptocurrencies
- Business process modeling

Some unconventional examples of applications: Sports tactics

Looking beyond the past: Analyzing the intrinsic playing style of soccer teams

Clijmans, J. et al., 2023

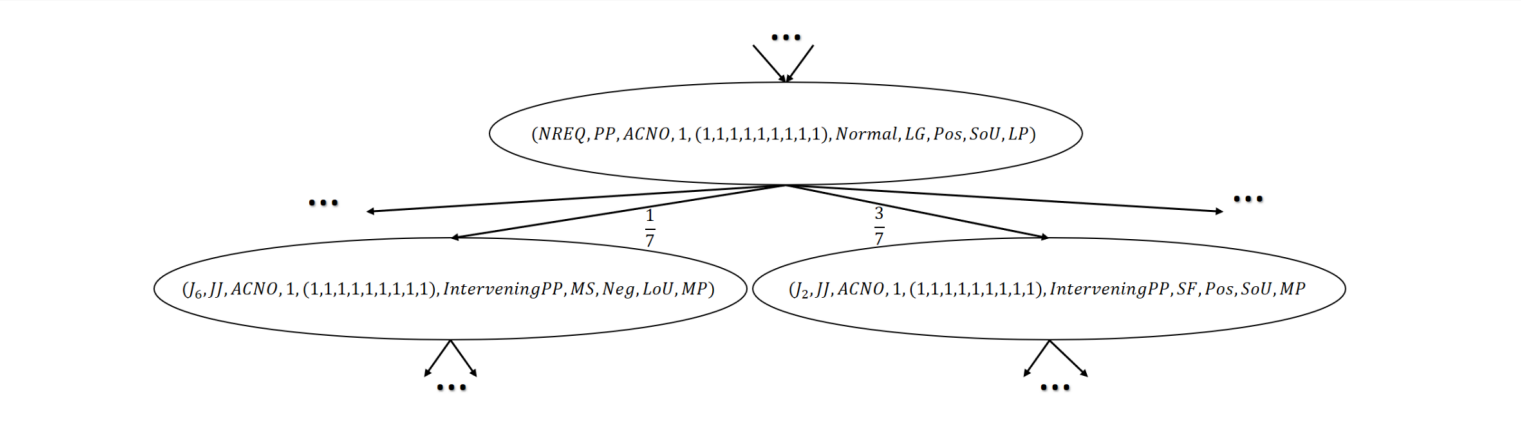


1. Learn an MDP model from event stream data
2. Apply probabilistic model checking with LTL to express outcomes of interest
3. Use the results to inform tactical decisions

Some unconventional examples of applications: Court interactions

Probabilistic model checking of temporal interaction dynamics in the supreme court

Das, S. and Sharma, A., 2023



1. Use transcripts for constructing a DTMRM
2. Formulate interesting queries over interactions by using PCTL
3. Verify queries by using a probabilistic model checker PRISM

PRISM: Practical Demo

We will now walk through two concrete PRISM models:

1. A **DTMC** — message delivery protocol with retries
2. An **MDP** — football club seasonal decisions

For each model we will:

- Write the model in the PRISM language
- Specify properties in PCTL
- Interpret the results

Example 1: Message Delivery Protocol (DTMC)

Scenario: A sender transmits a message that is lost with probability 0.3. The sender retries up to 3 times.

```
dtmc
module protokol
  s       : [0..2] init 0;
  retries : [0..3] init 0;

  [] s=0 & retries<3 ->
       0.7 : (s'=1) + 0.3 : (s'=0) & (retries'=retries+1);
  [] s=0 & retries=3 -> 1 : (s'=2);
  [] s=1 -> 1 : (s'=1);
  [] s=2 -> 1 : (s'=2);
endmodule
```

Example 1: Properties and Results

PCTL properties we verify:

Property	Question	Result
`P=? [ F "delivered" ]`	Probability of delivery?	0.973
`P=? [ F "failure" ]`	Probability of total failure?	0.027
`P=? [ F<=1 "delivered" ]`	Delivered on first try?	0.7
`P>=0.95 [ F "delivered"]`	Is reliability above 95%?	`true`

Note: `P=? [ F "failure" ]` $= 0.3^3 = 0.027$ — PRISM computes the **exact** mathematical value, not a simulation estimate.

Example 2: Football Club (MDP)

Scenario: Each season, the board chooses to *organise a tournament* or *do nothing*. The outcome is still probabilistic.

```
mdp
module club
  s : [0..1] init 0; // 0=struggling, 1=thriving

  [tournament] s=0 -> 0.6:(s'=1) + 0.4:(s'=0);
  [tournament] s=1 -> 0.9:(s'=1) + 0.1:(s'=0);
  [nothing]    s=0 -> 0.2:(s'=1) + 0.8:(s'=0);
  [nothing]    s=1 -> 0.6:(s'=1) + 0.4:(s'=0);
endmodule

label "struggling" = s=0;
label "thriving"   = s=1;
```

Example 2: Properties and Results

In an MDP we ask for **minimum and maximum** probabilities over all possible policies:

Property	**Question**	**Result**
`Pmax=? [ F "thriving" ]`	Best achievable probability?	1.0
`Pmin=? [ F "thriving" ]`	Worst case probability?	1.0
`Pmax=? [ X "thriving" ]`	Best prob. of thriving next season?	0.6
`Pmax>=0.9 [ F "thriving" ]`	Can we guarantee 90%?	`true`

The computed **optimal policy**: organise a tournament when struggling, do nothing when thriving — consistent with the analytical result from value iteration.

Conclusion

- Commonalities between different problem domains
- Considerable breadth and diversity to the set of applications
- Ease of rigorous modelling for stochastic systems
- Logical formalism for expressing quantitative properties allows going beyond classical notion of correctness typically used in formal verification
- Continually evolving tools and possible future domains of application